

Homework Assignment: University Database + LangGraph QA Agent

Objective

Design and implement a **question-answering system over a university database** using SQL and a **LangGraph-based agent**.

Your system should allow users to ask **natural language questions** about the database and receive accurate answers.

Functional Requirements

1. Database Design

Create a relational SQL database describing a university with the following core entities:

- **Teachers**
- **Students**
- **Courses**

The system must support the following relationships:

- A **Course** can be **taught by a Teacher in a specific semester**
- **Students can enroll in course offerings**
- Students receive **grades** for enrolled courses
- Each **Student** and **Teacher** must have a **name**

You may extend the schema if needed, but keep the design clean and explainable.

2. LangGraph Question Answering Agent

You must build a **LangGraph-based application** that allows users to ask questions about the database.

The agent should:

- Accept **natural language questions**
- Translate them into **SQL queries**
- Execute queries on the database
- Return **human-readable answers**

The agent must support **complex queries**, including:

- aggregations (e.g., averages, counts)
 - joins across multiple tables
 - filtering by semester, course, teacher, or student
 - multi-step reasoning
-

3. DB-Agnostic Design (Important Requirement)

Your LangGraph application should be designed to be **as database-agnostic as possible**.

This means:

- The core logic of the agent should **not depend on a specific database schema implementation**
 - It should be **easy to adapt** your agent to work with a **different database** (e.g., another schema)
 - Any schema-specific logic should be **isolated and modular**
-

4. Tracing and Observability (Mandatory)

You must implement **full tracing** of your LangGraph execution.

You may either:

- implement your **own tracing system**, or
- use **LangSmith**

However, you **must be able to clearly present and explain full run traces during the interview**.

Your system should show the full path from:

User Input → LangGraph Nodes → SQL → DB Results → Final Answer

The trace should make it easy to **debug and understand** how the system reached its output.

5. Debugging and Error Handling

Your system must be able to detect and handle problems (e.g., incorrect SQL generation, empty or inconsistent results, ambiguous user questions).

You should demonstrate how you **identify and debug issues** when the system produces incorrect output.

6. Unit Tests

You must include **unit tests** covering:

- database queries and joins
 - SQL generation from natural language
 - end-to-end behavior of the LangGraph agent
-

7. Code Quality Requirements

Your solution should be:

- clean
- modular
- well-structured
- easy to explain in an interview

You should structure your code so that each component has a **clear responsibility** (DB layer, agent logic, prompts, tracing, etc.).

8. Production Considerations

You must explain what would be required to bring your system to **production**, including topics such as:

- reliability
 - scalability
 - monitoring and tracing
 - security
 - deployment
-

Deliverables

You must submit:

1. SQL schema and seed data
2. LangGraph application code
3. Unit tests
4. Documentation explaining your design decisions
5. Example queries and outputs
6. Execution traces demonstrating the system flow
7. A **public GitHub repository** containing the above

You must publish the repository **before the technical interview**.

During the interview, you will be required to:

- present your system
 - walk through your code and architecture
 - demonstrate tracing
 - explain design decisions
 - defend your implementation choices
-

Evaluation Criteria

Your solution will be evaluated based on:

- correctness
 - clarity of design
 - ability to handle complex questions
 - quality of LangGraph usage
 - traceability and debuggability
 - code cleanliness and structure
 - ability to explain and defend your system
-

Notes

- The solution does **not need to be large**
 - It **must work**
 - It **must be explainable and debuggable**
 - You should be able to **walk through your design and trace execution step-by-step**
-

You may use **AI-powered coding assistants** (e.g., GitHub Copilot) to help with your implementation.

Good luck!